Fredrik Hammarberg

Optimising Weak Reference Processing in the JVM Z Garbage Collector

Reference Pipeline Optimisations and an Exploratory Weak-Field Mechanism for the JVM

UPPSALA
UNIVERSITET

Abstract

Reference processing can be a critical component of Java garbage collection latency, particularly in workloads with large numbers of weakly reachable objects. This thesis investigates how the Z Garbage Collector (ZGC) can reduce overhead when processing `WeakReference` objects that are not associated with a `ReferenceQueue`. Three orthogonal implementation strategies are developed in an OpenJDK research fork: a queue-less fast path that skips enqueue handling, a cache-friendlier dynamic-array representation for discovered references, and an optimised clear path that removes unnecessary per-reference work.

To evaluate these changes, a controlled benchmarking framework was constructed with repeated runs, CPU isolation, memory monitoring, and GC-log analysis. The evaluation compares all combinations of the three optimisation strategies against a baseline configuration using both single-object stress and multi-object phased-release workloads.

In addition to `WeakReference`-based optimisations, the thesis introduces an exploratory weak-field mechanism via a Java field annotation and corresponding VM support. This allows a representation-level comparison between object-based weak references and field-based weak semantics under the same experimental setup.

To be continued...

Acknowledgments

I would like to express my sincere gratitude to my supervisor for his guidance and support throughout this thesis work. It is through his help, encouragement, and expertise that I was able to learn so much about the JVM and bring this thesis to completion.

I would also like to thank my subject reviewer for his valuable academic guidance and thoughtful feedback, which helped strengthen the structure and quality of this thesis.

Additionally, I would like to thank the Department of Information Technology at Uppsala University for providing the resources necessary to conduct the performance evaluations and experiments described in this thesis.

Lastly, I would also like to thank my colleagues at Oracle for their insights and feedback on my work throughout the development process. Without their help, I would not have been able to achieve the results presented in this thesis.

Uppsala, Sweden, June 2026

Fredrik Hammarberg

Contents

List of Tables

List of Figures

List of Listings

1. Introduction

Java is one of the most widely deployed programming languages in enterprise and server-side computing [1], where applications routinely manage heaps of tens to hundreds of gigabytes. In these environments, the performance of the Java Virtual Machine's (JVM) garbage collector (GC) directly affects application latency and throughput. The Z Garbage Collector (ZGC) addresses this by performing nearly all garbage collection work concurrently with the application, achieving millisecond pause times regardless of heap size [2].

However, one aspect of ZGC that has received comparatively little optimisation attention is its processing of *weak references*. Java's `WeakReference` class allows applications to hold references that do not prevent the referent from being collected [3, 4], and it is used in cache-like structures such as `WeakHashMap` [5]. When the garbage collector determines that a weakly referenced object is no longer strongly reachable, it must clear the reference and, if the reference was registered with a `ReferenceQueue`, enqueue it for the application to process [6, 7].

The current ZGC reference processing pipeline treats all weak references uniformly: they are linked together in an intrusive linked list, processed, and then handed to a Java-level daemon thread (the `ReferenceHandler`) for enqueueing [8, 6]. This design introduces overhead that is disproportionate for a specific subset of weak references: those that are not registered with a `ReferenceQueue`.

This inefficiency has been previously identified in the OpenJDK bug tracker as JDK-8029205 [9], which reports the unnecessary pending-list traversal for references without a `ReferenceQueue`. However, no implementation addressing this issue has been integrated into the JDK to date. Building on that observation, this thesis investigates whether targeted modifications to ZGC's reference processing pipeline can reduce this overhead. Three orthogonal optimisation approaches are explored for `WeakReference`-based processing: skipping the enqueue path for references without queues, replacing the intrusive linked list with a cache-friendly dynamic array, and reducing per-reference instruction cost through an optimised clear path. In addition, an exploratory weak-field mechanism is studied as an alternative representation that avoids `WeakReference` object overhead in selected workloads.

1.1 Motivation

Weak references are handled very specifically in JVM garbage collection. Unlike regular references, which are handled implicitly by the mark phase,

weak references require explicit processing by the collector [8, 10]. Each discovered weak reference must be individually evaluated, its referent field cleared if the referent is unreachable, and the reference potentially enqueued. This per-reference processing cost scales linearly with the number of weak references, making it a bottleneck in applications that allocate millions of them.

The problem is amplified by a mismatch between the API design and its use cases. Java's `WeakReference` API provides an optional `ReferenceQueue` parameter [4], and the garbage collector's pipeline is designed around enqueue processing for cleared references [8, 6]. This assumption is still made even if the optional `ReferenceQueue` parameter was omitted. The result is a pipeline that performs unnecessary work for weak references that lack a `ReferenceQueue`.

ZGC's concurrent architecture makes this inefficiency particularly relevant. Because ZGC processes references concurrently with the application [2, 8], any reduction in per-reference overhead translates directly into less time spent in the reference processing phase. This results in less CPU time consumed by GC threads, ultimately improving application performance.

1.2 Problem Statement

The current ZGC reference processing pipeline does not distinguish between weak references with and without a `ReferenceQueue` during its phases [8]. All cleared references are threaded onto a pending list, transferred to the `ReferenceHandler` thread, and iterated regardless of whether they will actually be enqueued [6]. Additionally, the intrusive linked-list data structure used for discovered references exhibits poor cache locality [11].

This thesis seeks to determine whether these inefficiencies can be addressed through modifications to ZGC's reference processor, and to quantify the resulting performance impact.

1.3 Research Questions

The research questions that this thesis aims to address are:

RQ1 To what extent can modifications to ZGC's processing of `WeakReference` objects without a `ReferenceQueue` reduce the wall-clock time of reference processing, the total GC cycle time, the GC memory footprint, and the java heap size in select workloads?

RQ2 To what extent can expressing weak reachability through annotated object fields further reduce reference processing time, GC cycle time, GC memory footprint, and java heap usage compared to the optimised `WeakReference`-based approach of RQ1?

1.4 Scope and Delimitations

This thesis focuses on the reference processing pipeline within ZGC, specifically targeting weak references without a `ReferenceQueue`. The primary implementation work concerns `WeakReference`-based processing, with weak fields treated as an exploratory extension evaluated in the same benchmarking framework. Although the optimisation approaches are described in general terms the implementation and evaluation remain ZGC-specific. Furthermore, some of the optimisations, such as the optimised clear path, may have broader applicability beyond just weak references without a `ReferenceQueue`. However, these broader applications fall outside the scope of this thesis and are not evaluated.

The correctness of the optimisations is verified through the standard OpenJDK test suite and manual inspection; formal verification is outside the scope of this work. Performance measurements are conducted on a single hardware platform under controlled conditions and therefore may not generalise to all environments.

2. Background

The three optimisation approaches introduced in Chapter 1 rely on several key concepts and design decisions in the Java Virtual Machine's (JVM) garbage collection architecture, specifically the Z Garbage Collector (ZGC) and its reference processing pipeline. This chapter provides the necessary background on these topics to understand the motivation and implementation of the optimisations.

2.1 Garbage Collection in the JVM

The JVM provides automatic memory management through garbage collection (GC). Rather than requiring the programmer to explicitly allocate and free memory as in languages such as C or C++, the JVM tracks which objects are reachable from the application's root set and reclaims the memory of objects that are no longer reachable. The root set consists of all data that is directly accessible to the application, such as local variables on the stack, static fields, and active threads.

The JVM ships with several garbage collector implementations, each designed with different trade-offs between throughput, latency, and memory footprint [12]. The collectors are:

- **Serial GC** A single-threaded collector that performs all garbage collection work using one thread, avoiding inter-thread communication overhead. It is best suited to single-processor systems and small heaps.
- **Parallel GC** A throughput-oriented generational collector that uses multiple GC threads to speed up collection on multiprocessor systems. It is intended for medium to large data sets.
- **G1 GC (Garbage-First)** A mostly concurrent, region-based collector designed to scale from small to large multiprocessor machines. It targets predictable pause times with high probability while maintaining high throughput and is the default collector on most platforms [12].
- **ZGC (Z Garbage Collector)** A scalable, low-latency collector that performs expensive GC work concurrently with application threads. It targets pause times of only a few milliseconds (largely independent of heap size) and supports heap sizes from 8 MB to 16 TB.

This thesis focuses on ZGC, as its low-latency design and concurrent reference processing pipeline can particularly benefit from optimisations that reduce the amount of work done during reference processing.

2.1.1 Garbage Collection Phases

Garbage collection generally occurs multiple times (cycles) during the lifetime of a Java application. Each cycle consists of several distinct phases, each with specific responsibilities. The exact phases and their order can vary between collectors, but a common structure is as follows:

1. **Mark Phase** The collector identifies all live objects by traversing from the root set and marking all reachable objects. Dead objects are those that are not marked. This phase may be performed concurrently with the application or during a stop-the-world pause, depending on the collector design.
2. **Reference Processing Phase** After marking completes, the collector processes *Reference* objects discovered during the mark phase. This phase determines which references have dead referents and prepares them for enqueueing. Like marking, this phase may be concurrent or done during a pause.
3. **Sweep/Reclamation Phase** The collector reclaims memory occupied by dead objects. Some collectors use a sweep algorithm (scanning the heap and freeing unmarked objects) whilst others use evacuation (copying live objects to new memory regions).
4. **Compaction/Relocation Phase** Some collectors compact the heap to eliminate fragmentation, moving live objects to consolidate free space. Others relocate objects incrementally as part of the collection process.

The order and concurrency model of these phases varies significantly between collectors. Serial and Parallel GC typically perform all phases in stop-the-world pauses. G1 performs marking and sweeping mostly concurrently with short pauses for synchronisation. ZGC performs marking, reference processing, and relocation concurrently, using barriers to maintain consistency with concurrent application threads.

2.1.2 Generational Collection

All JVM garbage collectors (Serial, Parallel, G1, and Z) employ *generational collection*, based on the *weak generational hypothesis*: most objects die young and the ones that don't tend to live for a long time [13]. To employ this, the heap is divided into at least two generations:

- The **young generation**, where newly allocated objects reside. Young generation collections are frequent but fast, since most young objects are short-lived.
- The **old generation**, where objects that have survived multiple young collections are promoted to. Old generation collections are less frequent but more expensive since live objects are more plentiful here.

ZGC adopted generational collection starting with JDK 21 [14]. In generational ZGC, the young and old generations can be collected independently and

therefore concurrently. This is relevant to reference processing because references in different generations are handled during different collection cycles.

2.2 Java Reference Types

In addition to ordinary (strong) references, Java provides *reference objects* in the `java.lang.ref` package [3, 6] that allow applications to reference objects without preventing their garbage collection. These reference types are:

1. **SoftReference** The referent field is cleared at the discretion of the GC, typically when memory is low. Commonly used for memory-sensitive caches.
2. **WeakReference** The referent field is cleared as soon as it becomes weakly reachable (i.e., no strong or soft references exist to it). Widely used in caching frameworks, canonical mappings, and listener registrations.
3. **FinalReference** An internal JVM type (not part of the public API) used to implement the `finalize()` mechanism. Objects with finalisers are wrapped in a `FinalReference` so that the GC can schedule finalisation before reclaiming the object. This scheduling is done via a `ReferenceQueue`, which means that `FinalReference`s always have an associated `ReferenceQueue`.
4. **PhantomReference** The referent is never accessible through the reference; `get()` always returns `null`. Used for post-mortem cleanup actions, often to avoid the pitfalls of finalisation. Phantom references must be registered with a `ReferenceQueue` to be useful.

All four types extend the abstract class `Reference<T>` (in the `java.lang.ref` package), which defines the key fields of the reference object and the API for retrieving the referent, clearing the referent field, and checking if it is enqueued.

2.2.1 The Reference Object Lifecycle

Each `Reference` object maintains several fields that track its state [6]:

- `referent field` The referenced object. The GC may clear this field (set it to `null`) when the referent becomes eligible for collection according to the reference's type.
- `queue field` The `ReferenceQueue` this reference is registered with. If the reference was not created with a queue, this field points to the sentinel object `ReferenceQueue.NULL_QUEUE`.
- `next field` Used as a link pointer when the reference is enqueued in a `ReferenceQueue`.
- `discovered field` A field with dual purpose: during GC marking, it serves as a link in the garbage collector's internal *discovered list*; after

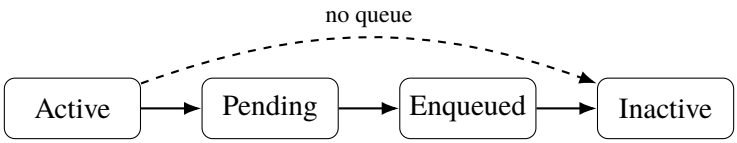

Figure 2.1. Lifecycle of `Reference` objects. References without a queue can bypass Pending and Enqueued.

processing, it is reused as a link in the VM's *pending list* that feeds the `ReferenceHandler` thread. Due to its first use, the GC can detect whether a reference has already been discovered by checking if this field is `null`.

A reference object transitions through the following states during its lifetime [15, 6]:

1. **Active** The reference has been created and the referent is still reachable (or not yet processed).
2. **Pending** The GC has determined the referent is eligible for clearing and has added the reference to the VM's pending list. The `ReferenceHandler` thread will process it.
3. **Enqueued** The `ReferenceHandler` thread has moved the reference into its associated `ReferenceQueue`, where the application can retrieve it via `poll()` or `remove()`.
4. **Inactive** The reference has been dequeued by the application, or was created without a queue and has been cleared. It is no longer tracked by the GC or queuing infrastructure.

An important observation is that references created *without* a `ReferenceQueue` can transition directly from Active to Inactive, bypassing the Pending and Enqueued states entirely. However, as discussed in Section 2.4, the default implementation does not take advantage of this shortcut. The lifecycle and this direct shortcut path are illustrated in Figure 2.1.

2.2.2 WeakReference and ReferenceQueue

A `WeakReference` provides two constructors (with and without a `ReferenceQueue`), as shown in Listing 1 and implemented in OpenJDK [4].

When a `WeakReference` is created with a `ReferenceQueue`, the application can be notified when the referent is collected. When created *without* a queue, the reference serves only as a conditional pointer, the application can call `get()` to check whether the referent is still alive, but receives no notification upon clearing.

The `ReferenceQueue` is implemented as a synchronized singly-linked list using the `next` field of the `Reference` objects. It provides `poll()` for non-blocking retrieval and `remove()` for blocking retrieval with an optional

```java
// Without a queue - queue field set to NULL_QUEUE
public WeakReference(T referent) {
    super(referent);
}

// With a queue
public WeakReference(T referent, ReferenceQueue<? super
    T> q) {
    super(referent, q);
}
```

Listing 1: WeakReference constructors

timeout [7]. This is the mechanism by which applications can react to the collection of referents, for example to clean up associated resources.

A key insight for this thesis is that Java explicitly supports queue-less weak references through the one-argument `WeakReference` constructor [4]. For workloads that only need liveness probing via `get()`, queue registration is unnecessary. In such cases, the VM still performs enqueue-path preparation work in the default pipeline, which motivated JDK-8029205 and the optimisation work in this thesis [9, 6, 8].

2.3 The Z Garbage Collector

ZGC is a concurrent, region-based, compacting garbage collector included in the JDK since JDK 11 (experimental) [2] and production-ready since JDK 15 [16]. Starting with JDK 21, ZGC supports generational collection, which is now the default mode [14]. ZGC's primary design goals are [2]:

- Sub-millisecond pause times that do not increase with heap size.
- Support for heap sizes from 8 MB to 16 TB.
- Concurrent marking, relocation, and reference processing.

ZGC achieves these goals through several key techniques:

Coloured Pointers.
ZGC embeds metadata in the unused bits of 64-bit object pointers. These *colour bits* encode information about the pointer's current state relative to the GC's phase, for example, whether the pointer has been remapped after relocation, or whether it points to a young or old generation object [2, 17]. This allows the GC to reason about pointer states without loading the object header.

Barriers.
ZGC uses barriers at every reference load or write. The barrier checks the colour bits of the loaded pointer and, if necessary, applies a fixup (e.g., remap-

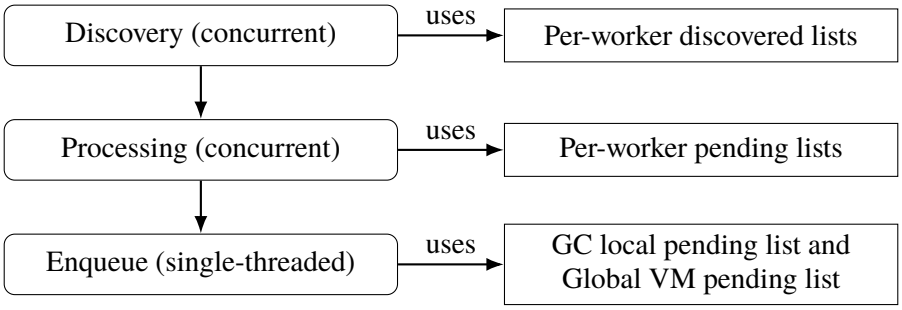

Figure 2.2. ZGC reference-processing pipeline and data flow from discovery to Java-level enqueue handling.

ping after relocation) [2, 18]. This is the mechanism that allows ZGC to perform most of its work concurrently with the application.

Concurrent Phases.
A ZGC collection cycle interleaves short stop-the-world pauses (for root scanning and synchronisation) with concurrent phases (for marking, reference processing, and relocation) [2, 8]. The concurrent phases run alongside application threads, using barriers to maintain consistency.

Multiple GC Worker Threads.
ZGC uses multiple GC worker threads to parallelize GC work. Each worker thread maintains its own data structures in many cases to avoid contention [19, 8].

2.4 Reference Processing Pipeline in ZGC

Reference processing is the mechanism by which the garbage collector handles `Reference` objects discovered during marking. The pipeline has three stages: *discovery*, *processing*, and *enqueueing*. In ZGC, the first two stages run concurrently with the application during the marking phase, while the enqueue stage involves handing off references to the Java-level `ReferenceHandler` thread [8, 6]. An overview of this data flow is shown in Figure 2.2.

2.4.1 Discovery

During the concurrent marking phase, when ZGC encounters a `Reference` object (an instance of `SoftReference`, `WeakReference`, `FinalReference`, or `PhantomReference`), it evaluates whether the reference should be *discovered*, i.e. added to a list for later processing.

The discovery decision depends on several factors:

1. **Is the reference active?** Only active references are eligible. For non-`FinalReference` types, an inactive reference has a `null` referent field. For `FinalReference`, inactivity is indicated by a non-null `next` field.
2. **Is the referent still strongly reachable?** If the referent is already marked as strongly live, the reference does not need processing. The garbage collector checks this via its liveness information.
3. **Soft reference policy.** For `SoftReference` objects, the collector applies a policy (based on available memory and the reference's last access time) to decide whether to keep the referent alive or clear the referent field [10, 20].
4. **Generational considerations.** In ZGC, only references residing in the old generation are discovered; young-generation references are always treated as strong [14, 8]. This is because reference processing introduces additional delay and uncertainty to the duration of the young-generation collection. By treating young references as strong, ZGC can complete young collections more predictably, which decreases the risk of the application running out of memory during a long young collection.

Once a reference is selected for discovery, it is recorded on a *discovered list*. In ZGC, each GC worker thread maintains its own discovered-list head, so discovery is implemented as a cheap, non-atomic prepend to a per-worker list [8]. This avoids CAS contention found in shared processors where multiple threads update the same discovered head [10].

2.4.2 Processing

After marking completes, the discovered lists are processed. For each discovered reference, the collector determines the final fate of the referent:

1. If the referent field has become `null` (cleared by the application concurrently), the reference is dropped from the list.
2. If the referent is now strongly reachable (e.g. it was marked as alive after the reference's discovery), the reference is dropped.
3. If the referent is still unreachable according to the reference's strength, the referent field is **cleared** (set to `null`), and the reference is prepared for enqueueing.

In ZGC, clearing involves setting the `referent` field to a coloured null pointer via compare-and-swap (CAS) to ensure thread safety since application threads may be concurrently accessing the reference [18, 8].

After clearing, the reference is linked onto the thread-local pending list. This involves writing the reference's address into the `discovered` field of the previously processed reference. Once the thread has processed all references in its discovered list, the thread-local list is atomically prepended to ZGC's internal pending list [8].

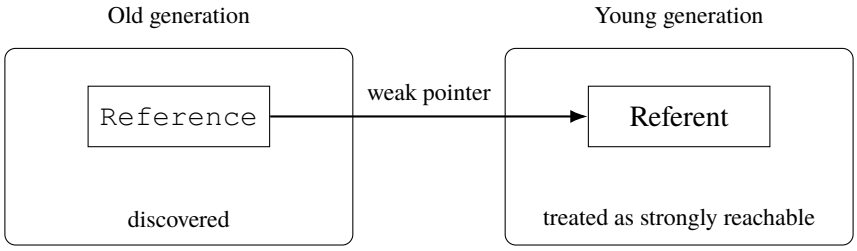

Figure 2.3. If an old-generation `Reference` points to a young referent, the young referent must remain alive during young collection.

Generational interactions are important during processing: when a `Reference` object is old but its referent is young, the referent is treated as strongly reachable and the referent field must not be cleared. Processing must therefore verify generation membership and, if appropriate, ensure young-generation liveness is preserved (by marking the young referent), as illustrated in Figure 2.3 [8, 21].

2.4.3 Enqueue

After a reference's referent field has been cleared, the reference must be made available to the application if it was created with a `ReferenceQueue`. This happens through the *pending list* mechanism which feeds the `ReferenceHandler` thread [8, 6]. The handoff process is as follows:

1. The GC has built an internal pending list by linking cleared references together via their `discovered` field.
2. Under the heap lock, the GC atomically prepends its internal pending list onto the global VM pending list.
3. The `ReferenceHandler` daemon thread is notified.
4. The `ReferenceHandler` thread wakes up, atomically retrieves the pending list, and iterates over it. For each reference, it checks whether `queue != NULL_QUEUE`. If the reference has a real queue, it calls `queue.enqueue(this)`; otherwise it simply advances to the next reference.

This handoff reveals an inefficiency: references created without a `ReferenceQueue` are still linked into the pending list, transferred to the `ReferenceHandler` thread, and iterated over only to be skipped [6, 8]. For workloads where the majority of weak references lack queues, this results in a considerable amount of extra work. This behaviour has been reported in an OpenJDK enhancement issue [9]. Avoiding enqueue-path work for references with `NULL_QUEUE` is one of the primary optimisation targets explored in this thesis.

3. Design and Implementation

This chapter presents the design rationale and implementation details of the four optimisation approaches investigated in this thesis. Each approach targets a specific source of overhead in ZGC's weak-reference processing pipeline, as motivated in Chapter 1. The first three approaches, namely skipping the enqueue path, replacing the intrusive linked list with a dynamic array, and optimising the clear path, operate within the existing `WeakReference` object model and were implemented as orthogonal modifications that can be independently enabled and combined. The fourth approach, weak fields, explores an alternative representation that avoids `WeakReference` objects entirely. For each approach, the underlying rationale is presented first, followed by the implementation details within the OpenJDK codebase. The complete changed source code is provided in Appendix A.

3.1 Skipping Enqueue Path

As described in Section 2.4.2, ZGC enqueues discovered references to the internal pending list regardless of whether they have a `ReferenceQueue` or not. This is inefficient for weak references without a `ReferenceQueue` as discussed in Section 2.4.3. Because of this, the reference processing pipeline was changed so that the enqueue path can be skipped for weak references without a `ReferenceQueue`. The intended effect of this optimisation is to both reduce the amount of work done by the GC threads during reference processing and to reduce the amount of work done by the `ReferenceHandler` thread when iterating over the pending list.

3.1.1 Implementation of Skipping Enqueue Path

In order to skip enqueueing weak references without a `ReferenceQueue` the `ReferenceProcessor` class was modified to check if the queue field of a weak reference being discovered is equal to the sentinel value `ReferenceQueue.NULL_QUEUE` before adding it to the discovered list [6, 8]. If the queue field is equal to `ReferenceQueue.NULL_QUEUE` (the weak reference was not created with a queue), it is added to a separate discovered list for weak references without a `ReferenceQueue` instead of being added to the regular discovered list. This separate discovered list is then processed

```
bool discover_reference(oop ref_obj, ReferenceType type) {
    zaddress ref = to_zaddress(ref_obj);

    zpointer* referent_field = reference_referent_addr(ref);
    zaddress referent = load_barrier(referent_field);
    if (!should_discover(ref, type, referent)) {
        return false;
    }

    if (type == REF_WEAK && !has_reference_queue(ref)) {
        weak_no_queue_list_per_worker.append(ref);
    } else {
        discovered_list_per_worker.append(ref);
    }

    mark_discovered(ref);
    return true;
}
```

Listing 2: Pseudocode of the null-queue fast path during discovery. Queue-less weak references are routed to a separate discovered list and processed by a no-enqueue pipeline. The corresponding implementation excerpts are provided in Appendix A.

independently by the GC threads and none of its weak references are enqueued to the pending list for the `ReferenceHandler` thread. Each GC thread maintains its own separate discovered list for weak references without a `ReferenceQueue` to avoid contention between threads.

The reason for separating the discovered lists instead of simply skipping enqueueing for weak references without a `ReferenceQueue` is that two unique reference processing functions can then be used for the two lists: one that includes enqueueing and one that does not. This allows for more efficient processing of the weak references without a `ReferenceQueue`, as it avoids unnecessary checks for whether the weak reference has a queue during processing.

This discovery-time split is summarised in Listing 2.

3.2 Replacing Intrusive Linked List with Dynamic Array

When processing the discovered references, ZGC iterates over the discovered list. This list is, as described in Section 2.4.1, implemented as an intrusive

linked list of the discovered references [8]. This iteration over a linked list is inefficient since it has poor cache locality [11]. Because of this, the intrusive linked list was replaced with a dynamic array for the discovered list for weak references without a `ReferenceQueue`. The intended effect of this optimisation is to reduce the amount of time spent loading the next reference to process and thereby allow for more efficient processing of the discovered references through better cache locality.

3.2.1 Implementation of Dynamic Array for Discovered References

The implemented dynamic array, called `ZWeakRefArray`, stores discovered references in a contiguous layout rather than linking them via the `discovered` field. The array is allocated on the C heap and grows using a power-of-two strategy (with a minimum capacity of 8), using `memcpy` for bulk copying during expansion rather than element-by-element loops (see Appendix A).

Each GC worker thread maintains its own `ZWeakRefArray` instance to avoid contention. During discovery, when a reference should be added to a discovery list, its data is appended to the worker's array instead of being appended to a linked list. The reference's `discovered` field is still set (to a self-referencing pointer) to mark the reference as discovered, preventing duplicate discovery.

During processing, the array is iterated sequentially using index-based access. Because the elements are stored contiguously in memory, this iteration improves cache line utilisation compared to following pointers scattered across the heap [11].

When using both dynamic arrays and separate discovered lists, only the discovered list for weak references without a `ReferenceQueue` is backed by a `ZWeakRefArray`, while the regular discovered list continues to use the linked list.

The resulting discovery and processing flow is summarised in Listing 3.

3.3 Optimised Clear Path

The standard reference processing path in ZGC uses general-purpose functions designed to handle all reference types uniformly. This generality introduces per-reference overhead that is unnecessary when processing only weak references without a `ReferenceQueue`, where the reference type is always `REF_WEAK` and enqueueing is never required.

The standard function for determining whether a reference's referent should be cleared performs several operations per reference [8, 18]:

1. A load barrier to read the referent from the reference object.

```
void discover_reference(oop ref_obj, ReferenceType type) {
    //...
    ZWeakRefData data;
    data.set_from_ref(ref);
    worker_array.append(data)
    //...
}

void process_worker_discovered_weak_refs_without_queue(ZWeakRefArray& arr) {
    size_t dropped = 0;
    for (size_t i = 0; i < arr.length(); i++) {
        const ZWeakRefData& data = arr.at(i);
        data.mark_not_discovered();

        if (try_make_inactive(data)) {
            cleared_weak_no_queue_count++;
        } else {
            dropped++;
        }
    }
    arr.clear_and_reserve(dropped);
}
```

Listing 3: Pseudocode for the `ZWeakRefArray`-backed queue-less weak-reference path. Discovery appends pre-fetched fields and processing scans the array sequentially. The corresponding source code is provided in Appendix A.

2. A virtual call to determine the reference type (soft, weak, final, or phantom).
3. A compare-and-swap (CAS) operation via a ZGC barrier to atomically set the referent field to a coloured null pointer.

For weak references without a `ReferenceQueue`, each of these operations can be simplified or eliminated:

1. When combined with the dynamic array optimisation, the referent field address and its value can be loaded during discovery and stored in the array entry, avoiding redundant memory loads during processing. This pre-loading is safe as long as the referent field is not set to a non-null value without any safeguards during processing. Since the possible concurrent application-level operations on `Reference` referents are clearing or enqueueing which result in nulling the referent field [6, 8].
2. The reference type is known to be `REF_WEAK` at the call site, eliminating the need for the virtual call to determine the type.
3. Since the only concurrent modification to the referent field is an application thread setting it to null [6, 18, 8], the CAS operation for clearing the referent field can be replaced with a simple store of a coloured null pointer without needing to check for concurrent updates.

3.3.1 Implementation of Optimised Clear Path

The implementation introduces a new function for determining whether a queue-less weak reference's referent should be cleared. This function takes a data struct (either from the dynamic array or constructed on the fly from the linked list) containing the pre-fetched field addresses and values.

The new function checks whether the referent is strongly live via metadata in ZGC's heap management. If the referent is not strongly live and resides in the old generation, its referent field is cleared using a direct pointer store rather than a CAS. This is safe under the constrained update model described above. If the referent is still live (either strongly marked or implicitly by residing in the young generation), the referent field is recoloured with the correct pointer metadata bits, this recolouring, however, has to occur with a CAS so that a null value is not overwritten with a non-null value. Additionally, if the referent is young and the young generation is currently in its mark phase, the referent is marked to preserve cross-generational liveness [8, 18, 21].

The `discovered` field is also cleared using a direct store to the field address rather than through an atomic access function, since no other thread modifies this field after discovery in this processing path [8].

This specialised inactive-check path is summarised in Listing 4.

```
bool try_make_inactive_fast(const ZWeakRefData& data) {
    // if used with the dynamic array
    zaddress referent = data.referent_addr;

    // if used with the linked list
    zaddress referent = load_barrier(data.
        referent_field_addr);

    // Young referents are implicitly kept alive by
        generational rules.
    if (is_young(referent)) {
        if (young_mark_is_active()) {
            mark_young_root(referent);
        }
        return false; // keep as active
    }

    // Old referent: clear only if not strongly live.
    if (!is_strongly_live(referent)) {
        *data.referent_field_addr = to_zpointer(zaddress
            ::null);
        return true; // became inactive
    }

    // Still live: recolour pointer metadata, but do not
        overwrite null.
    zpointer observed = data.referent_field_value;
    zpointer recoloured = remap_with_good_metadata(
        referent);
    cas_referent_if_non_null(data.referent_field_addr,
        observed, recoloured);
    return false;
}
```

Listing 4: Pseudocode for the specialised queue-less weak-reference inactive check. The old-generation clear path uses direct stores, while recolouring uses CAS to avoid null-to-non-null races. The corresponding implementation excerpts are provided in Appendix A.

3.4 Weak Fields

The three core optimisations above operate within the existing `WeakReference` object model. However, this model has an inherent structural overhead: each weak relation requires a separate Java object with queue and list-link metadata, and the GC must still discover and process that object. To answer Section 1.3, an exploratory weak-field mechanism was therefore implemented to test whether expressing weak reachability directly on object fields can reduce this overhead in weak-reference-dominated workloads.

3.4.1 Implementation of Weak Fields

At the Java level, a runtime-retained field annotation was added. During class-file parsing, annotated fields are marked as weak in field metadata.

On the VM side, weak-field support was integrated into ZGC's existing reference-processing framework (see Appendix A):

- Discovery support was added through a dedicated path that records candidate weak fields per worker thread.
- A specialised per-worker array (`ZWeakFieldArray`) stores field addresses and preloaded values to support sequential processing with low overhead.
- Processing uses ZGC barrier logic for weak semantics (preloaded weak-field clean barrier), clearing only when liveness rules permit.

In compiler and barrier plumbing, weak-annotated field loads are decorated as weak-reference accesses, so weak semantics are preserved consistently through generated code and runtime barriers.

4. Evaluation

This chapter describes the experimental setup used to evaluate the optimisation approaches presented in Chapter 3 and presents the results of the performance evaluation. Section 4.1 describes the build configurations used to isolate the effect of each optimisation flag. Section 4.2 details the benchmark design, execution environment, and measurement methodology. Finally, Section 4.3 presents the collected performance data.

4.1 Build Configurations

The three optimisation approaches are controlled by compile-time preprocessor flags: `ZUseOptimisedClearPath` for the optimised clear path, `ZUseSeparateDiscoveredLists` for the separate discovered list, and `ZUseDynamicArray` for the dynamic array. Each flag can be independently enabled (1) or disabled (0) via compiler flags passed during the JDK build.

Eight build configurations were created to evaluate all combinations of the three optimisation flags, as shown in Table 4.1. Each configuration produces a separate JDK build with its own binary, allowing direct comparison under identical benchmark conditions.

Table 4.1. *Build configurations and their enabled optimisation flags*

Configuration	Clear Path	Sep. Lists	Dyn. Array
`none`	—	—	—
`dyn_only`	—	—	✓
`sep_only`	—	✓	—
`sep_dyn`	—	✓	✓
`clear_path_only`	✓	—	—
`clear_path_dyn`	✓	—	✓
`clear_path_sep`	✓	✓	—
`all`	✓	✓	✓

Weak fields were evaluated as a separate configuration in the same benchmark harness. In this setup, the `WeakReference` benchmark class is replaced by its weak-field counterpart while using the baseline (`none`) ZGC optimisation flag configuration. This isolates representation-level effects from the three `WeakReference` pipeline optimisations, whilst still allowing direct comparison within the same execution and analysis workflow.

4.2 Performance Evaluation

To evaluate the performance impact of the optimisation approaches, four microbenchmarks were used under controlled conditions: two `WeakReference`-based benchmarks and two weak-field counterparts with analogous workload structure. The benchmarks and their execution environment are described below.

4.2.1 Benchmark Design

Two benchmark families were used to evaluate the optimisation approaches. The first family isolates reference-processing cost around a single shared referent, and the second is a more realistic multi-object workload that introduces greater allocation and liveness variability. Each family was implemented both with `WeakReference` objects and with weak fields. In order to enable `WeakReference`-to-weak-field comparison, both benchmark families use holder objects that either contain a `WeakReference` or an annotated weak field.

Single-Object Benchmark

The Single-Object benchmarks allocate 10 million holder objects with either a `WeakReference` object or an annotated weak field each that all point to a single target object. The holder objects are allocated and stored in a randomised order using a Fisher-Yates shuffle [22] to avoid artificial locality patterns. None of the weak references are created with a `ReferenceQueue`. After a brief holding period, the single strong reference to the target object is released and a GC cycle is triggered manually. Because all weak references become clearable simultaneously in a single GC cycle, this benchmark maximises the per-cycle reference processing load and isolates the cost of the processing pipeline itself.

Multi-Object Benchmark

The multi-object benchmarks are a slightly more realistic workload. It allocates approximately 7.3 million objects, each containing a randomly sized byte array payload (between 509 and 1097 bytes). Each object has a strong reference and a corresponding holder object with either a `WeakReference` object or an annotated weak field. 5% of the `WeakReference`s are created with a `ReferenceQueue` and 95% without, reflecting a workload dominated by queue-less weak references but where the enqueue path is still exercised for a minority of references. The allocation and storing order of both strong references and holder objects are randomised using separate Fisher-Yates shuffles [22] to avoid artificial locality patterns.

Unlike the Single-Object benchmark, the strong references are not all released at once. Instead, the benchmark proceeds through five rounds of clearing. In each round, 20% of the strong references are nulled in a randomised or-

der, making the corresponding objects eligible for garbage collection. Between rounds, the benchmark sleeps to allow GC cycles to discover and process the weak references whose referent now is unreachable. This gradual release pattern means that weak references are discovered and processed across multiple GC cycles rather than in a single burst.

The benchmark runs three iterations with a five-second cooldown between them to allow the JVM to stabilise. Each iteration performs the full allocation, clearing, and GC cycle sequence independently.

4.2.2 Execution Environment

To ensure reproducible measurements, the benchmarks were executed under controlled conditions using a dedicated runner script. The script applies the following isolation measures:

- **CPU pinning**: The Java process was pinned to a fixed set of CPU cores using `taskset`. Meanwhile, the monitoring and scripting processes ran on separate cores to avoid interference.
- **CPU frequency control**: The CPU governor is set to performance mode at the base clock frequency. Both the minimum and maximum scaling frequencies are locked to the base frequency to prevent frequency scaling from introducing variance.
- **Process priority**: The Java process is run with elevated priority (`nice -n -5`) to reduce scheduling interference from other system processes.
- **Cache clearing**: Filesystem caches are dropped between runs by writing to `/proc/sys/vm/drop_caches`.
- **Cooldown period**: A configurable cooldown period (default 10 seconds) is inserted between variant runs to allow the system to return to a stable state.

For the single-object benchmark, the JVM was configured with a fixed 7 GB heap and ZGC enabled. Tenuring thresholds are set to 1 to promote the target object to the old generation quickly, ensuring that the reference processor discovers it. GC statistics and reference processing logs were enabled to collect the results from.

For the multi-object benchmark the JVM was configured with a fixed 8 GB heap, ZGC enabled, and a forced major collection interval of 0.5 seconds to ensure frequent GC activity during the benchmark. Tenuring thresholds are set to 1 to promote objects to the old generation quickly, ensuring that the reference processor discovers them. This is necessary since ZGC doesn't perform reference processing in the young generation. GC statistics and reference processing logs were enabled to collect the results from.

Table 4.2. *Continuous monitoring metrics used in the measurement pipeline.*

Metric	Units	Description
Timestamp	ms	Sample timestamp used to align and aggregate runs over time.
RSS	kB	Process resident set size (RSS), representing total physical memory in use by the JVM process.
GC Committed Memory	kB	GC-related memory currently committed by the JVM (actually backed memory from Native Memory Tracking).
Java Heap	kB	Java heap usage sampled from `jcmd GC.heap_info`.
Phase	label	Benchmark phase label emitted by the benchmark and used for phase-aligned analysis.
Iteration	label	Inner-iteration identifier (multi-object benchmark) used to align repeated phase sequences.

4.2.3 Measurement Methodology

Each benchmark was run for 100 outer iterations. Within each outer iteration, all eight `WeakReference` variants are run in sequence with environment preparation between each. In the same outer iteration, the weak-field benchmark variant is then executed as an additional run. This interleaved ordering mitigates systematic drift in system conditions affecting one configuration more than others.

During each run, a memory monitoring script samples memory metrics from the process at high frequency. Additionally, the current benchmark phase is tracked and for the multi-object benchmark the inner iteration is tracked as well. This tracking is done from the benchmark's log file to enable phase-aligned analysis. See Table 4.2 for the memory metrics collected.

In addition to the continuous memory samples, GC statistics are parsed from the log files of each run. For each GC metric, the parser extracts the Total Avg/Max pair and unit reported by ZGC. For the single-object benchmark, Max is used as the primary value for comparison; for the multi-object benchmark, Avg is used. The analysis pipeline also recognises the weak-field run naming convention and includes weak-field results as a dedicated `weak_fields` configuration in comparative plots and summary tables. See Table 4.3 for the GC log metrics collected.

Table 4.3. *GC log metrics used in the measurement pipeline.*

Metric	Units	Description
Major Collection	ms	End-to-end duration of major collections reported by GC statistics.
Old Generation	ms	Total time spent in old-generation collection work.
Old Subphase: Concurrent Mark Follow	ms	Time spent in old-generation concurrent mark-follow processing.
Old Subphase: Concurrent References Process	ms	Time spent in concurrent reference processing in old-generation collection.
Young Subphase: Concurrent Mark Follow	ms	Time spent in young-generation concurrent mark-follow processing.

4.3 Presentation of Results

This section presents the empirical results of the performance evaluation. The comparison covers the baseline (`none`) and all seven non-empty combinations of the three optimisation flags (clear path, separate lists, dynamic array) for `WeakReference`-based processing. In addition, a dedicated `weak_fields` configuration is included, where the weak-field benchmark classes are executed in the same harness and analysed with the same tooling. This configuration set enables two complementary comparisons:

- **Intra-model comparison** among `WeakReference` variants to isolate the impact of each optimisation and their interactions.
- **Inter-model comparison** between optimised `WeakReference` processing and weak-field representation to evaluate structural overhead trade-offs.

The primary performance metrics are the GC timing statistics extracted from ZGC logs, including major collection time, old-generation collection time, concurrent mark-follow subphases, and concurrent reference-processing subphases. Memory behaviour is characterised using continuous samples of RSS, GC committed memory, and Java heap usage. Following the methodology in Section 4.2.3, single-object benchmark comparisons use the per-run maximum values as primary statistics, while multi-object comparisons use per-run averages. This choice reflects the fact that single-object runs are designed to create concentrated peak load in one collection event, whereas multi-object runs distribute work over repeated phases.

Figures 4.1 and 4.2 show box plots of the GC timing metrics for the single-object and multi-object benchmarks. They summarise the distribution of per-run maxima for the single-object benchmark and per-run averages for the multi-object benchmark, allowing comparison of both central tendency and variability. Figures 4.3 and 4.4 show representative continuous-memory traces

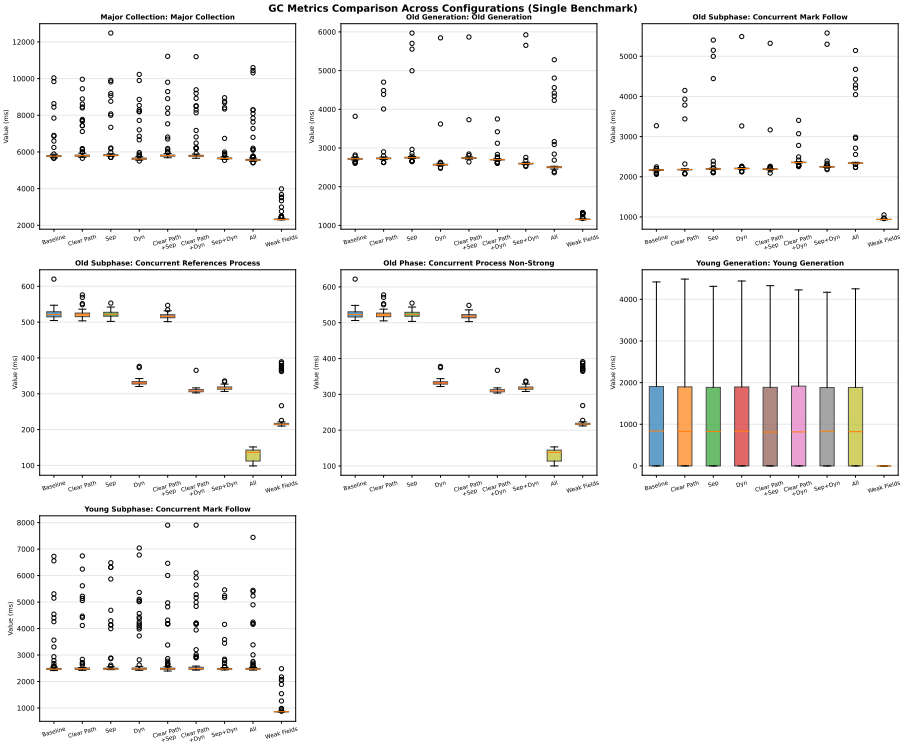

Figure 4.1. Box plots of GC metrics across evaluated configurations for the single-object benchmark.

for the same benchmark families, illustrating how memory usage evolves over time and across GC phases.

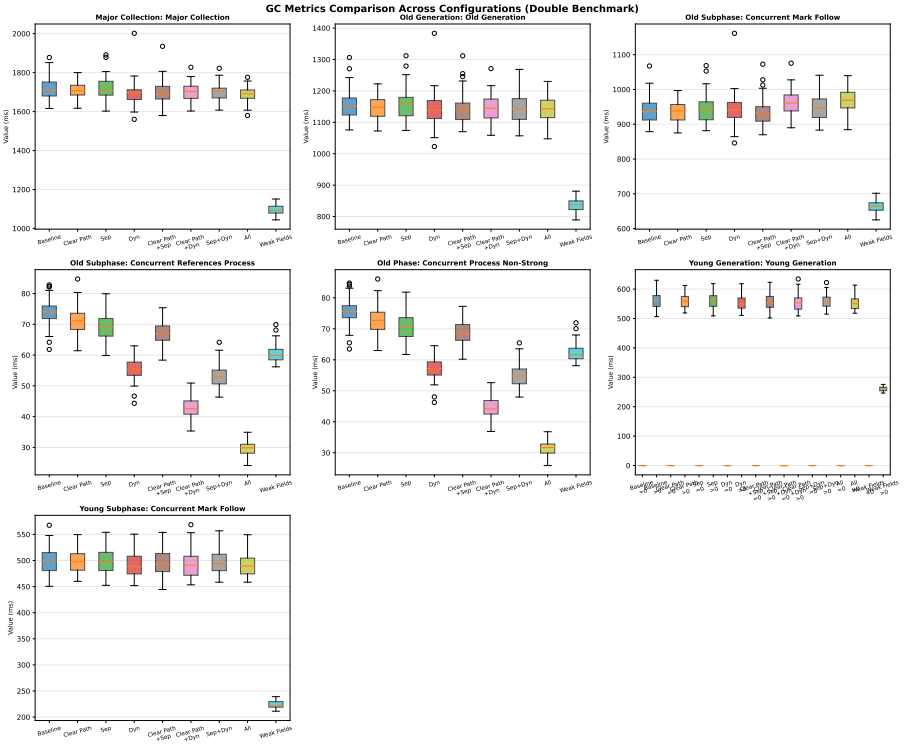

Figure 4.2. Box plots of GC metrics across evaluated configurations for the multi-object benchmark.

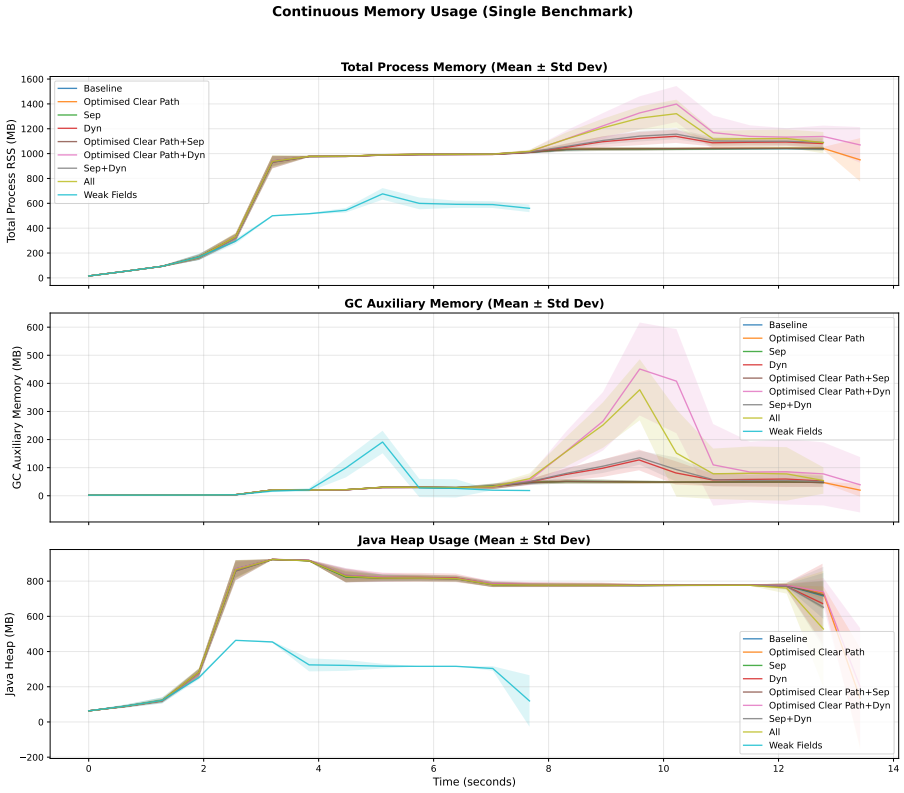

Figure 4.3. Continuous memory trace for the single-object benchmark.

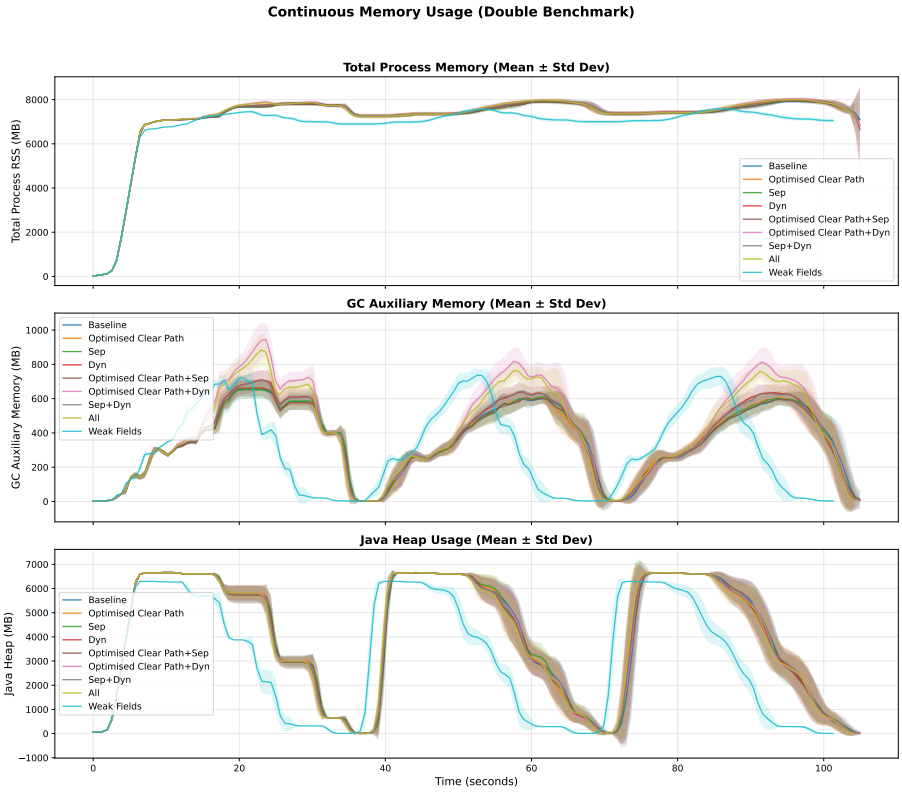

Figure 4.4. Continuous memory trace for the multi-object benchmark.

5. Analysis of Results

6. Related Work

7. Discussion

7.1 Future Work

Can the optimisations developed for ZGC be applied to other garbage collectors in the JVM, such as the G1 garbage collector, and if so, what performance improvements can be observed?

8. Conclusion

References

[1] Wikipedia, "Programming languages used in most popular websites." https://en.wikipedia.org/wiki/Programming_languages_used_in_most_popular_websites, 2025. Accessed: 2026-03-12.

[2] OpenJDK, "Jep 333: Zgc: A scalable low-latency garbage collector (experimental)." https://openjdk.org/jeps/333, 2018. Accessed: 2026-03-12.

[3] Oracle, "java.lang.ref (java se 25)." https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ref/package-summary.html, 2025. Accessed: 2026-03-12.

[4] OpenJDK, "Openjdk source: java.lang.ref.weakreference.java." https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/lang/ref/WeakReference.java, 2026. GitHub code blob. Accessed: 2026-03-24.

[5] OpenJDK, "Openjdk source: java.util.weakhashmap.java." https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/util/WeakHashMap.java, 2026. GitHub code blob. Accessed: 2026-03-24.

[6] OpenJDK, "Openjdk source: java.lang.ref.reference.java." https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/lang/ref/Reference.java, 2026. GitHub code blob. Accessed: 2026-03-24.

[7] OpenJDK, "Openjdk source: java.lang.ref.referencequeue.java." https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/lang/ref/ReferenceQueue.java, 2026. GitHub code blob. Accessed: 2026-03-24.

[8] OpenJDK, "Openjdk source: Zgc reference processor (zreferenceprocessor.cpp)." https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zReferenceProcessor.cpp, 2026. GitHub code blob. Accessed: 2026-03-24.

[9] OpenJDK Bug System, "Unnecessary pending-list traversal for references without referencequeue." https://bugs.openjdk.org/browse/JDK-8029205, n.d. Accessed: 2026-02-24.

[10] OpenJDK, "Openjdk source: Shared reference processor (referenceprocessor.cpp)." https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/shared/referenceProcessor.cpp, 2026. GitHub code blob. Accessed: 2026-03-24.

[11] T. M. Chilimbi, B. Davidson, and J. R. Larus, "Cache-conscious structure layout," in *Proceedings of the ACM SIGPLAN 1999 Conference on Programming Language Design and Implementation (PLDI)*, pp. 1–12, ACM, 1999.

[12] Oracle, "Available collectors." https://docs.oracle.com/en/java/javase/25/gctuning/available-collectors.html, 2025. Java Platform, Standard Edition 25 Garbage Collection Tuning Guide. Accessed: 2026-02-24.

[13] H. Lieberman and C. Hewitt, “A real time garbage collector based on the lifetimes of objects,” Tech. Rep. AIM-569a, MIT Artificial Intelligence Laboratory, Oct. 1981. Accessed: 2026-02-24.
[14] OpenJDK, “Jep 439: Generational zgc.” https://openjdk.org/jeps/439, 2023. Accessed: 2026-03-12.
[15] Oracle, “Reference (java se 25).” https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ref/Reference.html, 2025. Accessed: 2026-03-12.
[16] OpenJDK, “Jep 377: Zgc: A scalable low-latency garbage collector (production).” https://openjdk.org/jeps/377, 2020. Accessed: 2026-03-12.
[17] OpenJDK, “Openjdk source: Zgc address representation (zaddress.hpp).” https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zAddress.hpp, 2026. GitHub code blob. Accessed: 2026-03-24.
[18] OpenJDK, “Openjdk source: Zgc barrier implementation (zbarrier.cpp).” https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zBarrier.cpp, 2026. GitHub code blob. Accessed: 2026-03-24.
[19] OpenJDK, “Openjdk source: Zgc worker infrastructure (zworkers.cpp).” https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zWorkers.cpp, 2026. GitHub code blob. Accessed: 2026-03-24.
[20] OpenJDK, “Openjdk source: Reference policy (referencepolicy.cpp).” https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/shared/referencePolicy.cpp, 2026. GitHub code blob. Accessed: 2026-03-24.
[21] OpenJDK, “Openjdk source: Generational zgc implementation (zgeneration.cpp).” https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zGeneration.cpp, 2026. GitHub code blob. Accessed: 2026-03-24.
[22] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3rd ed., 1997. Section 3.4.2, Algorithm P.

Appendix A.
Changed Source Code

This appendix lists the source files changed to implement the weak-reference processing optimisations evaluated in this thesis. The code shown is from the `all` configuration variant, which combines all three optimisations (separate discovered lists, dynamic arrays, and the optimised clear path). Each configuration variant is stored under `patches/<variant>/src/` and overlaid onto the upstream OpenJDK source tree before building.

Files changed in the `all` variant:

- `patches/all/src/hotspot/share/gc/z/zWeakRefArray.hpp`
- `patches/all/src/hotspot/share/gc/z/zReferenceProcessor.hpp`
- `patches/all/src/hotspot/share/gc/z/zReferenceProcessor.cpp`

Common files shared by all ref-proc variants:

- `patches/base/src/hotspot/share/gc/z/zStat.hpp`
- `patches/base/src/hotspot/share/gc/z/zStat.cpp`

Additional files for variants with separate discovered lists:

- `patches/sep_base/src/hotspot/share/gc/shared/collectedHeap`
- `patches/sep_base/src/hotspot/share/gc/z/zCollectedHeap.cpp`
- `patches/sep_base/src/hotspot/share/gc/z/zCollectedHeap.hpp`
- `patches/sep_base/src/hotspot/share/gc/z/zGeneration.hpp`
- `patches/sep_base/src/hotspot/share/gc/z/zGeneration.inline`
- `patches/sep_base/src/hotspot/share/runtime/threads.cpp`

Dynamic-array Data Structure

File: `patches/all/src/hotspot/share/gc/z/zWeakRefArray.hpp`

```
1  /*
2   * Copyright (c) 2025, Oracle and/or its affiliates. All
       rights reserved.
3   * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE
       HEADER.
4   *
5   * This code is free software; you can redistribute it
      and/or modify it
6   * under the terms of the GNU General Public License
      version 2 only, as
```

```
 * published by the Free Software Foundation.
 *
 * This code is distributed in the hope that it will be useful, but WITHOUT
 * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
 * FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
 * version 2 for more details (a copy is included in the LICENSE file that
 * accompanied this code).
 *
 * You should have received a copy of the GNU General Public License version
 * 2 along with this work; if not, write to the Free Software Foundation,
 * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
 *
 * Please contact Oracle, 500 Oracle Parkway, Redwood Shores, CA 94065 USA
 * or visit www.oracle.com if you need additional information or have any
 * questions.
 */

#ifndef SHARE_GC_Z_ZWEAKREFARRAY_HPP
#define SHARE_GC_Z_ZWEAKREFARRAY_HPP

#include "gc/z/zAddress.hpp"
#include "memory/allocation.hpp"
#include "utilities/debug.hpp"
#include "utilities/powerOfTwo.hpp"
#include "utilities/globalDefinitions.hpp"
#include <string.h>

struct ZWeakRefData {
  zpointer* referent_field_addr;
  zaddress* discovered_field_addr;
  zaddress referent_addr;
  zpointer referent_field_value;
};


class ZWeakRefArray : public AnyObj {
private:
  ZWeakRefData* _data;
```

```
45    const size_t _ZWeakRefData_size = sizeof(ZWeakRefData)
          ;
46    size_t _length;
47    size_t _capacity;
48
49    void expand_to(size_t new_capacity) {
50      assert(new_capacity > _capacity, "expected growth
            but %ld <= %ld", new_capacity, _capacity);
51
52      ZWeakRefData* new_data = (ZWeakRefData*)AllocateHeap
            (new_capacity * _ZWeakRefData_size, mtGC);
53
54      if (_length > 0) {
55        // Use memcpy for trivially copyable types - much
              faster than element-by-element copy
56        memcpy(new_data, _data, _length *
              _ZWeakRefData_size);
57      }
58
59      if (_data != nullptr) {
60        FreeHeap(_data);
61      }
62
63      _data = new_data;
64      _capacity = new_capacity;
65    }
66
67    void grow(size_t min_capacity) {
68      size_t new_capacity = MAX2((size_t) 8,
            next_power_of_2(min_capacity - 1));
69      expand_to(new_capacity);
70    }
71
72  public:
73    ZWeakRefArray() : _data(nullptr), _length(0),
          _capacity(0) {}
74
75    ~ZWeakRefArray() {
76      if (_data != nullptr) {
77        FreeHeap(_data);
78        _data = nullptr;
79      }
80    }
81
82    void append(const ZWeakRefData& entry) {
83      if (_length >= _capacity) {
84        grow(_length + 1);
```

```
85      }
86      _data[_length] = entry;
87      _length++;
88    }
89
90    // Get entry at index
91    const ZWeakRefData& at(size_t index) const {
92      assert(index < _length, "index out of bounds: %ld (
            length: %ld)", index, _length);
93      return _data[index];
94    }
95
96    // Current length
97    size_t length() const {
98      return _length;
99    }
100
101   // Whether the array is empty
102   bool is_empty() const {
103     return _length == 0;
104   }
105
106   // Current capacity
107   size_t capacity() const {
108     return _capacity;
109   }
110
111   void clear_and_reserve(size_t new_capacity) {
112     if (new_capacity == 0) {
113       // Special case for clearing to empty - free
              memory and reset
114       if (_data != nullptr) {
115         FreeHeap(_data);
116         _data = nullptr;
117       }
118       _length = 0;
119       _capacity = 0;
120       return;
121     }
122     _length = 0;
123     if (_data != nullptr) {
124       FreeHeap(_data);
125     }
126     _capacity = MAX2((size_t) 8, next_power_of_2(
          new_capacity - 1));
127     _data = (ZWeakRefData*)AllocateHeap(_capacity *
          _ZWeakRefData_size, mtGC);
```

```
  }

  // Clear without deallocating
  void clear() {
    _length = 0;
  }

  // Reserve capacity without clearing
  void reserve(size_t new_capacity) {
    if (new_capacity > _capacity) {
      grow(new_capacity);
    }
  }
};

#endif // SHARE_GC_Z_ZWEAKREFARRAY_HPP
```

Reference Processor Interface

File: `patches/all/src/hotspot/share/gc/z/zReferenceProcessor.hpp`

```
/*
 * Copyright (c) 2015, 2024, Oracle and/or its affiliates. All rights reserved.
 * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
 *
 * This code is free software; you can redistribute it and/or modify it
 * under the terms of the GNU General Public License version 2 only, as
 * published by the Free Software Foundation.
 *
 * This code is distributed in the hope that it will be useful, but WITHOUT
 * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
 * FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
 * version 2 for more details (a copy is included in the LICENSE file that
 * accompanied this code).
 *
 * You should have received a copy of the GNU General Public License version
```

```
16  * 2 along with this work; if not, write to the Free
        Software Foundation,
17  * Inc., 51 Franklin St, Fifth Floor, Boston, MA
        02110-1301 USA.
18  *
19  * Please contact Oracle, 500 Oracle Parkway, Redwood
        Shores, CA 94065 USA
20  * or visit www.oracle.com if you need additional
        information or have any
21  * questions.
22  */
23
24 #ifndef SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
25 #define SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
26
27 #include "gc/shared/referenceDiscoverer.hpp"
28 #include "gc/z/zAddress.hpp"
29 #include "gc/z/zWeakRefArray.hpp"
30 #include "gc/z/zValue.hpp"
31
32 class ConcurrentGCTimer;
33 class ReferencePolicy;
34 class ZWorkers;
35
36
37
38 class ZReferenceProcessor : public ReferenceDiscoverer {
39   friend class ZReferenceProcessorTask;
40
41 private:
42   static const size_t ReferenceTypeCount = REF_PHANTOM +
          1;
43   typedef size_t Counters[ReferenceTypeCount];
44
45   ZWorkers* const             _workers;
46   ReferencePolicy*            _soft_reference_policy;
47   bool
         _uses_clear_all_soft_reference_policy;
48   ZPerWorker<Counters>        _encountered_count;
49   ZPerWorker<Counters>        _discovered_count;
50   ZPerWorker<Counters>        _enqueued_count;
51   ZPerWorker<size_t>
         _encountered_weak_refs_without_queue_count;
52   ZPerWorker<size_t>
         _discovered_weak_refs_without_queue_count;
53   ZPerWorker<size_t>
         _cleared_weak_refs_without_queue_count;
```

```
54    ZPerWorker<zaddress>        _discovered_list;
55    ZContended<zaddress>        _pending_list;
56    zaddress                    _pending_list_tail;
57    ZPerWorker<ZWeakRefArray>
          _discovered_weak_refs_without_queue_arr;
58    ZPerWorker<bool>            _array_empty;
59    OopHandle                   _null_queue_handle;
60
61    bool is_inactive(zaddress reference, oop referent,
          ReferenceType type) const;
62    bool is_strongly_live(oop referent) const;
63    bool is_softly_live(zaddress reference, ReferenceType
          type) const;
64
65    bool should_discover(zaddress reference, ReferenceType
           type, oop referent) const;
66    bool try_make_inactive(zaddress reference,
          ReferenceType type) const;
67    bool try_make_inactive_fast(const ZWeakRefData& data);
68
69    void discover(zaddress reference, ReferenceType type,
          zaddress referent);
70
71    void verify_empty() const;
72
73    void process_worker_discovered_list(zaddress
          discovered_list);
74    void process_worker_discovered_weak_refs_without_queue
          (ZWeakRefArray& weak_refs_without_queue);
75    void work();
76    void collect_statistics();
77
78    zaddress swap_pending_list(zaddress pending_list);
79
80    inline bool has_reference_queue(zaddress reference);
81
82    void initialize_null_queue_handle();
83
84  public:
85    ZReferenceProcessor(ZWorkers* workers);
86
87
88    void set_soft_reference_policy(bool
          clear_all_soft_references);
89    bool uses_clear_all_soft_reference_policy() const;
90
91    void reset_statistics();
```

```
92
93    virtual bool discover_reference(oop reference,
          ReferenceType type);
94    void process_references();
95    void enqueue_references();
96
97    void verify_pending_references();
98
99    void initializeResources();
100 };
101
102 #endif // SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
```

Reference Processor Implementation (Excerpts)

Excerpt: `zReferenceProcessor.cpp` (constructor and per-worker storage setup)

```
1     : _workers(workers),
2       _soft_reference_policy(nullptr),
3       _uses_clear_all_soft_reference_policy(false),
4       _encountered_count(),
5       _discovered_count(),
6       _enqueued_count(),
7       _encountered_weak_refs_without_queue_count(),
8       _discovered_weak_refs_without_queue_count(),
9       _cleared_weak_refs_without_queue_count(),
10      _discovered_list(zaddress::null),
11      _pending_list(zaddress::null),
12      _pending_list_tail(zaddress::null),
13      _discovered_weak_refs_without_queue_arr(),
14      _array_empty(),
15      _null_queue_handle() {
16
17    _array_empty.set_all(true);
18    log_info(gc, ref)("Reference Processor created, Config
          : All optimisations enabled");
19  }
20
21  void ZReferenceProcessor::set_soft_reference_policy(bool
        clear_all_soft_references) {
22    static AlwaysClearPolicy always_clear_policy;
23    static LRUMaxHeapPolicy lru_max_heap_policy;
24
25    _uses_clear_all_soft_reference_policy =
          clear_all_soft_references;
```

```
26
27    if (clear_all_soft_references) {
28      _soft_reference_policy = &always_clear_policy;
29    } else {
30      _soft_reference_policy = &lru_max_heap_policy;
31    }
32
33    _soft_reference_policy->setup();
34  }
35
36  bool ZReferenceProcessor::
        uses_clear_all_soft_reference_policy() const {
37    return _uses_clear_all_soft_reference_policy;
38  }
```

Excerpt: `zReferenceProcessor.cpp` (discovery path with queue-less weak-reference routing)

```
1
2     // Update statistics
3     if (type == REF_WEAK && !has_reference_queue(reference
          )) {
4       _discovered_weak_refs_without_queue_count.get()++;
5     }
6     else {
7       _discovered_count.get()[type]++;
8     }
9
10    assert(ZHeap::heap()->is_old(reference), "Must be old"
          );
11    assert(is_null(reference_discovered(reference)), "
          Already discovered");
12
13    if (type == REF_WEAK && !has_reference_queue(reference
          )) {
14      zpointer* const referent_addr =
            reference_referent_addr_non_vol(reference);
15      zaddress* const discovered_addr =
            reference_discovered_addr(reference);
16      const zpointer referent_value = *referent_addr;
17
18      // WeakReference with null ReferenceQueue - remember
            for special processing
19      ZWeakRefArray& weak_refs_without_queue =
            _discovered_weak_refs_without_queue_arr.get();
20      ZWeakRefData data = {referent_addr, discovered_addr,
            referent, referent_value};
21      weak_refs_without_queue.append(data);
```

```
22      _array_empty.set(false);
23      reference_set_discovered(reference, reference); //
            mark as discovered
24    } else {
25      if (type == REF_FINAL) {
26        // Mark referent (and its reachable subgraph)
              finalizable. This avoids
27        // the problem of later having to mark those
              objects if the referent is
28        // still final reachable during processing.
29        volatile zpointer* const referent_addr =
              reference_referent_addr(reference);
30        ZBarrier::mark_barrier_on_old_oop_field(
              referent_addr, true /* finalizable */);
31      }
32
33      // Add reference to discovered list
34      zaddress* const head = _discovered_list.addr();
35      reference_set_discovered(reference, *head);
36      *head = reference;
37    }
38  }
39
40  bool ZReferenceProcessor::discover_reference(oop
        reference_obj, ReferenceType type) {
41    if (!RegisterReferences) {
42      // Reference processing disabled
43      return false;
44    }
45
46    log_trace(gc, ref)("Encountered Reference: "
          PTR_FORMAT " (%s)", p2i(reference_obj),
          reference_type_name(type));
47
48    const zaddress reference = to_zaddress(reference_obj);
49
50    // Update statistics
51    if (type == REF_WEAK && !has_reference_queue(reference
          )) {
52      _encountered_weak_refs_without_queue_count.get()++;
53    }
54    else {
55      _encountered_count.get()[type]++;
56    }
57
58    if (ZHeap::heap()->is_young(reference)) {
```

```
59      // Don't discover young references. Young gen
            reference processing is scary and therefore not
            supported.
60      return false;
61    }
62
63    volatile zpointer* const referent_addr =
          reference_referent_addr(reference);
64    const zaddress ref_raw_addr = ZBarrier::
          load_barrier_on_oop_field(referent_addr);
65    const oop referent = to_oop(ref_raw_addr);
66
67    if (!should_discover(reference, type, referent)) {
68      // Not discovered
69      return false;
70    }
71
72    discover(reference, type, ref_raw_addr);
73
74    // Discovered
75    return true;
76  }
77
78  void ZReferenceProcessor::process_worker_discovered_list
        (zaddress discovered_list) {
79    zaddress keep_head = zaddress::null;
80    zaddress keep_tail = zaddress::null;
81
82    // Iterate over the discovered list and unlink them as
          we go, potentially
83    // appending them to the keep list
```

Excerpt: `zReferenceProcessor.cpp` (processing paths for separate lists and dynamic-array variants)

```
1         log_trace(gc, ref)("\"Enqueued\" Weak Reference
              without Queue");
2         // Update statistics
3         _cleared_weak_refs_without_queue_count.get()++;
4       } else {
5         log_trace(gc, ref)("Dropped Weak Reference without
              Queue");
6         dropped++;
7       }
8       SuspendibleThreadSet::yield();
9     }
10    weak_refs_without_queue.clear_and_reserve(dropped);
11  }
```

```
12
13
14
15 void ZReferenceProcessor::work() {
16   SuspendibleThreadSetJoiner sts_joiner;
17
18   ZPerWorkerIterator<zaddress> iter(&_discovered_list);
19   ZPerWorkerIterator<ZWeakRefArray> iter_weak_refs(&
         _discovered_weak_refs_without_queue_arr);
20   ZPerWorkerIterator<bool> iter_array_empty(&
         _array_empty);
21
22   zaddress* list_addr = nullptr;
23   ZWeakRefArray* array_addr = nullptr;
24   bool* array_empty = nullptr;
25
26   for (; iter.next(&list_addr) && iter_weak_refs.next(&
         array_addr) && iter_array_empty.next(&array_empty)
         ;) {
27
28     const zaddress discovered_list = AtomicAccess::xchg(
           list_addr, zaddress::null);
29     const bool has_array = !AtomicAccess::xchg(
           array_empty, true);
30     const bool has_discovered = discovered_list !=
           zaddress::null;
31
32     if (has_discovered) {
33       process_worker_discovered_list(discovered_list);
34     }
35     if (has_array) {
36       process_worker_discovered_weak_refs_without_queue
             (*array_addr);
37     }
38   }
39 }
40
41 void ZReferenceProcessor::verify_empty() const {
42 #ifdef ASSERT
43   ZPerWorkerConstIterator<zaddress> iter(&
         _discovered_list);
44   for (const zaddress* list; iter.next(&list);) {
45     assert(is_null(*list), "Discovered list not empty");
46   }
47
48   ZPerWorkerConstIterator<ZWeakRefArray> iter_weak_refs
         (&_discovered_weak_refs_without_queue_arr);
```

```
49    for (const ZWeakRefArray* array; iter_weak_refs.next(&
          array);) {
50      assert(array->is_empty(), "Discovered weak refs
            without queue not empty");
51    }
52    assert(is_null(_pending_list.get()), "Pending list not
          empty");
53  #endif
54  }
55
56  void ZReferenceProcessor::reset_statistics() {
57    verify_empty();
```

Excerpt: `zReferenceProcessor.cpp` (queue sentinel initialisation and queue checks)

```
1     }
2
3     const zaddress ref_queue_addr_value = ZBarrier::
          load_barrier_on_oop_field(reference_queue_addr(
          reference));
4     const oop ref_queue = to_oop(ref_queue_addr_value);
5     bool result = ref_queue != _null_queue_handle.resolve
          ();
6     return result;
7   }
8
9   void ZReferenceProcessor::initialize_null_queue_handle()
        {
10    EXCEPTION_MARK;
11    TempNewSymbol class_name = SymbolTable::new_symbol("
          java/lang/ref/ReferenceQueue");
12    Klass* k = SystemDictionary::resolve_or_fail(
          class_name, true, CHECK);
13    InstanceKlass* ik = InstanceKlass::cast(k);
14    ik->initialize(CHECK);
15    fieldDescriptor fd;
16    bool found = ik->find_local_field(SymbolTable::
          new_symbol("NULL_QUEUE"),
17                                      vmSymbols::
                                            referencequeue_signature
                                            (), &fd);
18    assert(found && fd.is_static(), "ReferenceQueue.
          NULL_QUEUE missing");
19    oop null_q = ik->java_mirror()->obj_field(fd.offset())
          ;
20    _null_queue_handle = OopHandle(Universe::vm_global(),
          null_q);
```

```
21 }
22
23 void ZReferenceProcessor::initializeResources() {
24   Thread* const current = Thread::current();
25   guarantee(current->is_Java_thread(), "Must be called
       by a JavaThread");
26   guarantee(!current->is_ConcurrentGC_thread(), "Must
       not be called by a GC thread");
27
28   initialize_null_queue_handle();
29 }
```